

Memory Management

CSE 3331: Operating Systems

Sajid Fardin Dipto
Lecturer
Department of CSE
Northern University Bangladesh

Reference book

- Operating System Concepts by Avi Silberschatz (9th edition)
- Chapter -8

Outline

- Memory Management
- Main Memory
- Basic Hardware
- Address Binding
- Logical Versus Physical Address Space
- Dynamic Loading

Memory Management

- In previous, we showed how the CPU can be shared by a set of processes. As a result of CPU scheduling, improving both the utilization of the CPU and the speed of the computer's response to its users.
- To realize this increase in performance, however, we must keep several processes in memory—that is, we must share memory.

Memory Management

- Various ways to manage memory
- Memory management algorithms
  - Primitive bare machine approach
  - Paging strategies
  - Segmentation strategies

Memory Management

- Memory Management
  - Main Memory
  - Virtual Memory

Basic Hardware

- Various ways to manage memory
- Memory management algorithms
  - Primitive bare machine approach
  - Paging strategies
  - Segmentation strategies

Memory Management

- Memory consists of a large array of bytes, each with its own address.
- The CPU fetches instructions from memory according to the value of the program counter.
- A **typical instruction-execution** cycle,
  - First **fetches** an instruction from memory.
  - The instruction is then **decoded** and may cause operands to be fetched from memory.
  - After the instruction has been **executed** on the operands, results may be stored back in memory.

Basic Hardware

- CPU can directly access:
  - Registers built in processors
  - Main memory
- There are machine instructions that take memory addresses as arguments, but none that take disk addresses.
- Therefore, any instructions in execution, and any data being used by the instructions, must be in one of these direct-access storage devices.
- If the data are not in memory, they must be moved there before the CPU can operate on them.

Basic Hardware

- Registers that are built into the CPU are generally accessible within one cycle of the CPU clock.
- The same cannot be said of main memory, which is accessed via a transaction on the memory bus. Completing a memory access may take many cycles of the CPU clock.
- In such cases, the processor normally needs to stall, since it does not have the data required to complete the instruction that it is executing.

Basic Hardware

- This situation is intolerable because of the frequency of memory accesses. The **remedy** is to add fast memory between the CPU and main memory, typically on the CPU chip for fast access. So a **cache** was introduced.
- For proper system operation we must protect the operating system from access by user processes. On multiuser systems, we must additionally protect user processes from one another. This protection must be provided by the hardware.

Basic Hardware

- We first need to make sure that each process has a separate memory space. Separate per-process memory space protects the processes from each other and is fundamental to having multiple processes loaded in memory for concurrent execution.
- To separate memory spaces, we need the ability to determine the **range of legal addresses** that the process may access and to ensure that the process can access only these legal addresses. We can provide this protection by using two registers,
  - ❑A base register &
  - ❑A limit register

illustrated in Figure 8.1.

Basic Hardware

- Base register: holds the smallest legal physical memory address;
- Limit register: specifies the size of the range.

- For example, if the base register holds 300040 and the limit register is 120900, then the program can legally access all addresses from 300040 through 420939 (inclusive).

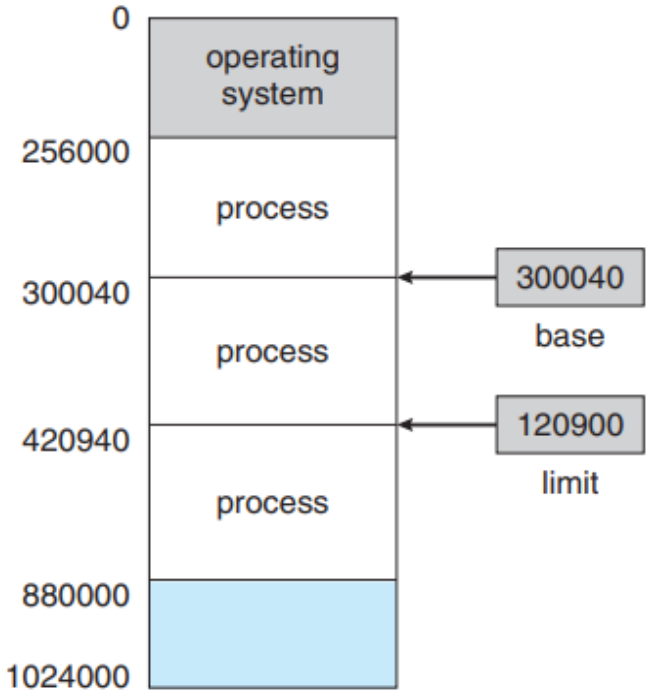


Figure 8.1 A base and a limit register define a logical address space.

Basic Hardware

- Protection of memory space is accomplished by having the CPU hardware compare every address generated in user mode with the registers. Any attempt by a program executing in user mode to access operating-system memory or other users' memory results in a trap to the operating system, which treats the attempt as a fatal error (Figure 8.2).

Basic Hardware

- This scheme prevents a user program from (accidentally or deliberately) modifying the code or data structures of either the operating system or other users.

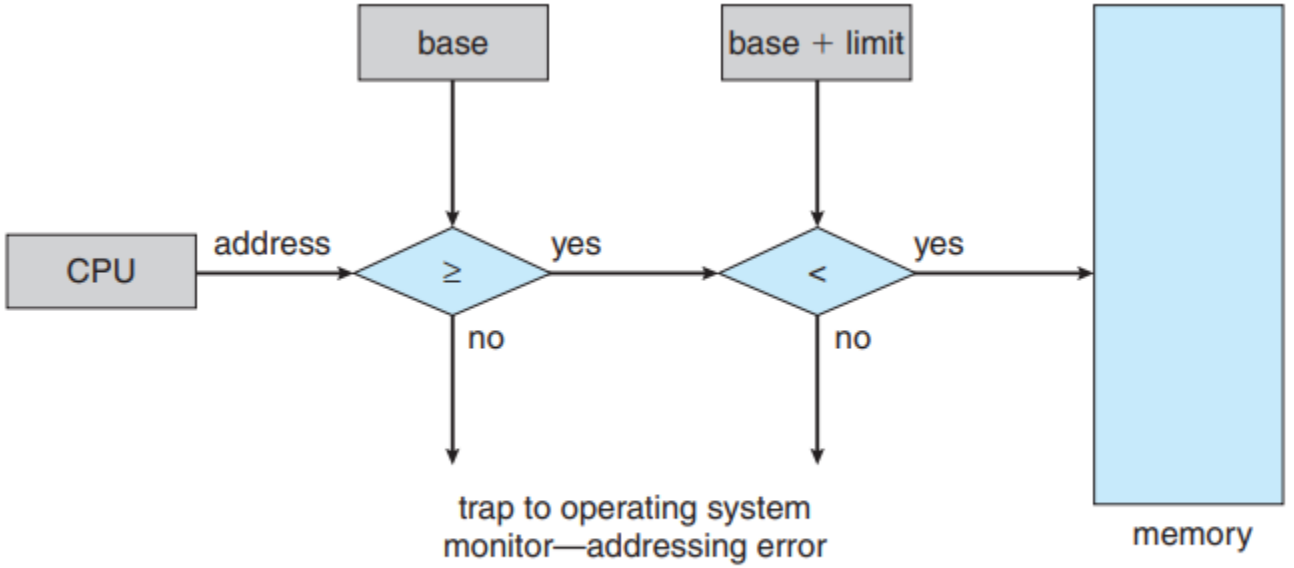


Figure 8.2 Hardware address protection with base and limit registers.

Basic Hardware

- The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction.
- Since privileged instructions can be executed only in kernel mode, and since only the operating system executes in kernel mode, only the operating system can load the base and limit registers.
- This scheme allows the operating system to change the value of the registers but prevents user programs from changing the registers' contents.

Address Binding

- Usually, a program resides on a disk as a binary executable file.
- To be executed, the program must be brought into memory and placed within a process.
- Depending on the memory management in use, the process may be moved between disk and memory during its execution.
- The processes on the disk that are waiting to be brought into memory for execution form the input queue.
- Input Queue → Select a process → Load it into memory → Execute and access instructions and data from memory → Process terminates → Memory space is declared available.

Address Binding

- Most systems allow a user process to reside in any part of the physical memory.
- Though the address space of the computer may start at 00000, the first address of the user process need not be 00000.
- In most cases, a user program goes through several step (Compile time, load time, execution time). Addresses may be represented in different ways during these steps.

Address Binding

- Source program: Addresses are generally symbolic (such as the variable count).
- Compiler : It typically binds these symbolic addresses to relocatable addresses (such as “14 bytes from the beginning of this module”)
- Linkage editor or loader : Binds the relocatable addresses to absolute addresses (such as 74014).
- Each binding is a mapping from one address space to another.
- The Address Binding refers to the mapping of computer instructions and data to physical memory locations.

Address Binding

Classically, the binding of instructions and data to memory addresses can be done at any step along the way:

- Compile time.
- Load time
- Execution time

Compile time

- **Compile time.** If you know at compile time where the process will reside in memory, then **absolute code** can be generated.
- For example, if you know that a user process will reside starting at location R, then the generated compiler code will start at that location and extend up from there.
- If, at some later time, the starting location changes, then it will be necessary to recompile this code.

Load time

- **Load time**. If it is not known at compile time where the process will reside in memory, then the compiler must generate **relocatable code**.
- In this case, final binding is delayed until load time. If the starting address changes, we need only reload the user code to incorporate this changed value.

Execution/Run time

- **Execution time.** If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.
- Special hardware must be available for this scheme to work. Most general-purpose operating systems use this method.

Logical Versus Physical Address Space

- Logical address : An address generated by the CPU.
- Physical address : An address seen by the memory unit—that is, the one loaded into the memory-address register of the memory.
- The **compile-time and load-time** address-binding methods generate **identical** logical and physical addresses.
- However, the execution-time address-binding scheme results in differing logical and physical addresses. In this case, we usually refer to the logical address as a **virtual address**.

Logical Versus Physical Address Space

- Logical address space : The set of all logical addresses generated by a program.
- Physical address space: The set of all physical addresses corresponding to these logical addresses.
- In the execution-time address-binding scheme, the logical and physical address spaces differ.
- The run-time mapping from virtual to physical addresses is done by a hardware device called the **memory-management unit (MMU)**.

Logical Versus Physical Address Space

- Dynamic relocation using a relocation register

- The base register is now called a relocation register. The value in the relocation register is added to every address generated by a user process at the time the address is sent to memory.

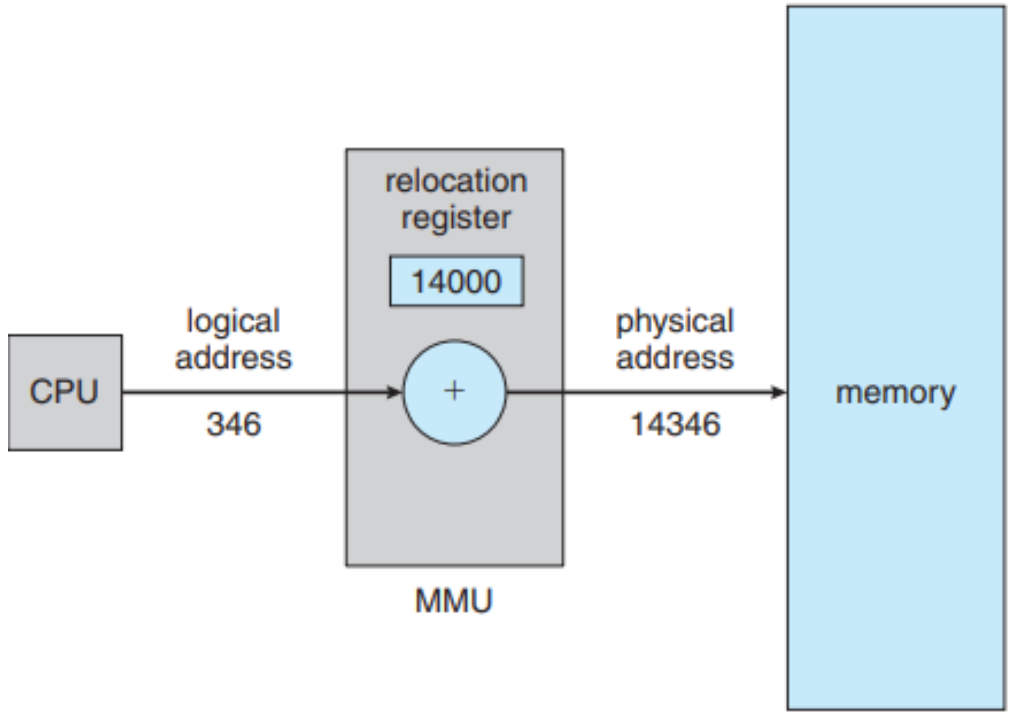


Figure 8.4 Dynamic relocation using a relocation register.

Logical Versus Physical Address Space

- Dynamic relocation using a relocation register

- For example, if the base is at 14000, then an attempt by the user to address location 0 is dynamically relocated to location 14000; an access to location 346 is mapped to location 14346.

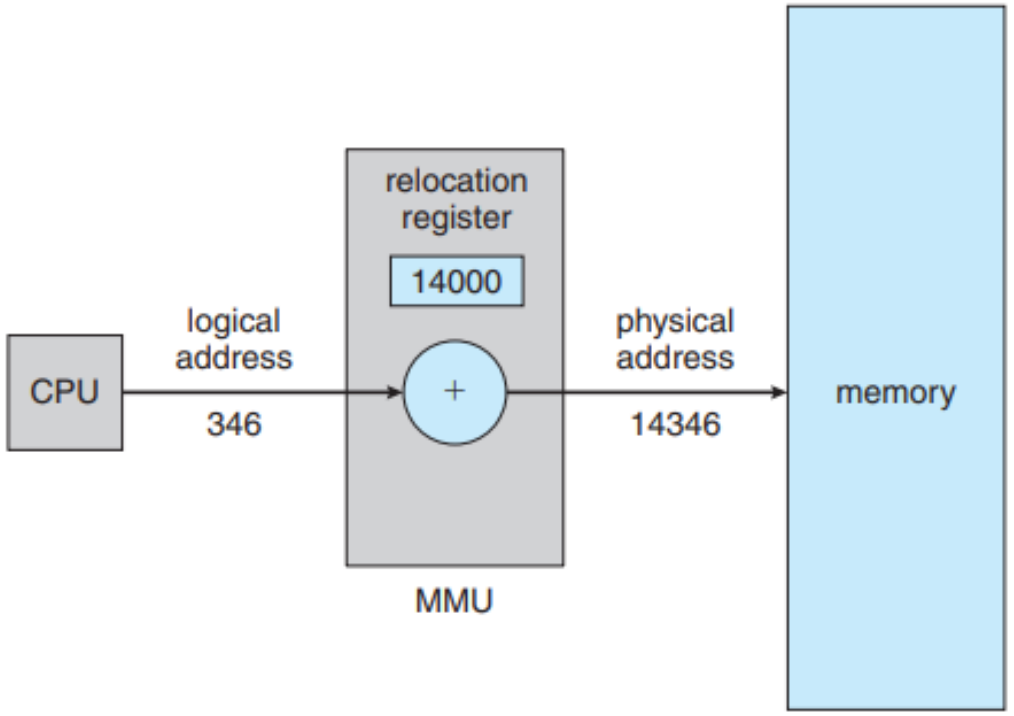


Figure 8.4 Dynamic relocation using a relocation register.

Logical Versus Physical Address Space

- The user program never sees the real physical addresses. The user program deals with logical addresses. The memory-mapping hardware converts logical addresses into physical addresses.
- We have two different types of addresses:
  - logical addresses (in the range 0 to max)
  - physical addresses (in the range R + 0 to R + max for a base value R).
- The user program generates only logical addresses and thinks that the process runs in locations 0 to max.
- However, these logical addresses must be mapped to physical addresses before they are used.

Dynamic loading

- In our discussion so far, it has been necessary for the entire program and all data of a process to be in physical memory for the process to execute.
- The size of a process has thus been limited to the size of physical memory.
- To obtain better memory-space utilization, we can use dynamic loading. With dynamic loading, a routine is not loaded until it is called. All routines are kept on disk in a relocatable load format.

Dynamic loading

- The main program is loaded into memory and is executed. When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded. If it has not, the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change.
- Then control is passed to the newly loaded routine.

Advantage of Dynamic loading

- The advantage of dynamic loading
  - An unused routine is never loaded.
  - This method is particularly useful when large amounts of code are needed to handle infrequently occurring cases, such as error routines.
  - Although the total program size may be large, the portion that is used (and hence loaded) may be much smaller.
- Dynamic loading does not require special support from the operating system. It is the responsibility of the users to design their programs to take advantage of such a method.

Swapping

- A process must be in memory to be executed.
- A process, however, can be swapped temporarily out of memory to a backing store and then brought back into memory for continued execution (Figure 8.5).

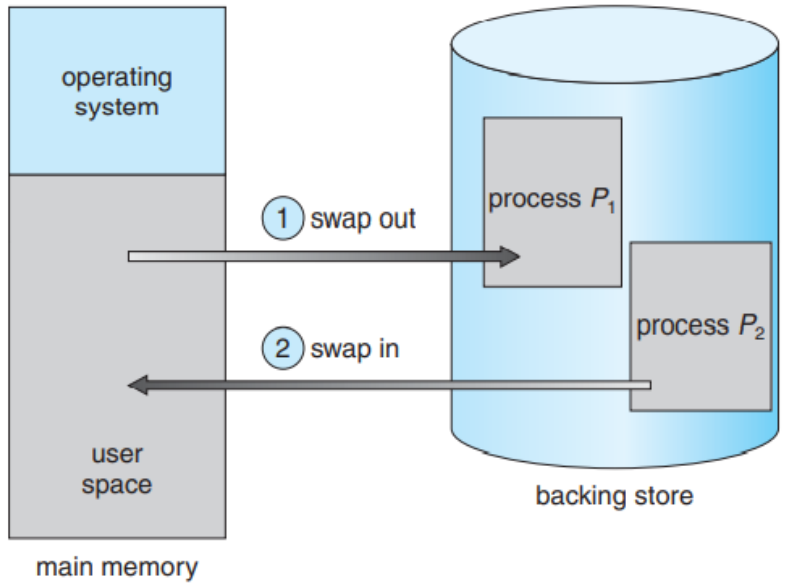


Figure 8.5 Swapping of two processes using a disk as a backing store.

Swapping

- **Example:**

  In a multiprogramming environment with a round-robin CPU scheduling algorithm, when a quantum expires, the memory manager will start to swap out the process that just finished and to swap another process into the memory space that has been freed.

- Normally, a process that is swapped out will be swapped back into the same memory space it occupied previously. But it depends on the address binding method.

Swapping

- If binding is done at assembly or load time: the process can not be easily moved to a different location.
- If binding is done at execution time: the process can be swapped into a different memory space, because the physical addresses are computed during execution time.

Swapping

- **Backing store**:
  - Swapping requires a backing store(commonly a fast disk).
  - It must be large enough to accommodate copies of all memory images for all users.
  - It must provide direct access to these memory images.

Swapping Time

- The context-switch time in such a swapping system is fairly high.
- Let,
- The user process = 100 MB
- the backing store transfer rate = 50 MB per second.
- The actual transfer of the 100-MB process to or from main memory takes

  =100 MB/50 MB per second

  = 2 seconds
- The swap time is 200 milliseconds. Since we must swap both out and in, the total swap time is about 4,000 milliseconds

Swapping Time

- For efficient CPU utilization, we want the execution time for each process to be long relative to the swap time.
- Thus in a round robin CPU scheduling algorithm, for example, the time quantum should be substantially larger than 0.516 seconds.
- The major part of the swap time is transfer time
- The total transfer time is directly propositional to the amount of memory swapped.

Swapping Time

- A process must be completely idle in order to be swapped.
- A process may be waiting for an I/O operation when we want to swap that process to free up memory.
- If the I/O is asynchronously accessing the user memory for I/O buffers, then the process can not be swapped.

Backing store

- The system maintains a ready queue consisting of all processes whose memory images are on the backing store or in memory and are ready to run.
- Whenever the CPU scheduler decides to execute a process, it calls the dispatcher.
- The dispatcher checks to see whether the next process in the queue is in memory.
- If it is not, and if there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the desired process.
- It then reloads registers and transfers control to the selected process.

Backing store

- Swapping makes it possible for the total physical address space of all processes to exceed the real physical memory of the system, thus increasing the degree of multiprogramming in a system.

Memory Allocation

- Memory Allocation:
  - Contiguous memory allocation
    - Fixed size partitioning
    - Variable size partitioning
  - Non contiguous memory allocation

Memory Allocation

How to allocate memory?

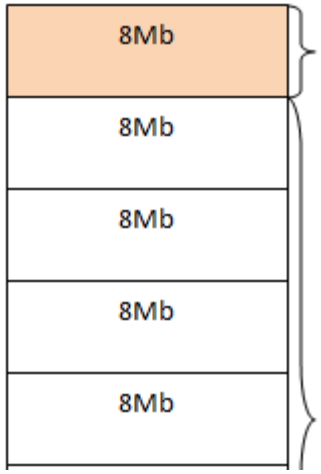
- Divide memory into several fixed-sized partitions.
- Each partition may contain exactly one process.
- When a partition is free, a process is selected from the input queue and is loaded into the free partition.
- When the process terminates, the partition becomes available for another process.

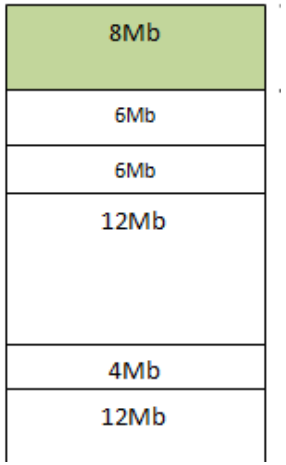
Memory Allocation

- Memory partitioning is the system by which the memory of a computer system is divided into sections for use by the programs. These memory divisions are known as partitions.
- Contiguous memory allocation
  - Fixed size partitioning
  - Variable size partitioning

Memory Allocation

- Fixed size partitioning: One of the simplest methods for allocating memory is to divide memory into several fixed-sized partitions. Each partition may contain exactly one process.





- When a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process.

Memory Allocation

- Variable size partitioning: In the variable-partition scheme, the operating system keeps a table indicating which parts of memory are available and which are occupied. Initially, all memory is available for user processes and is considered one large block of available memory, a hole. Eventually, as you will see, memory contains a set of holes of various sizes.

Memory Allocation

- Dynamic storage allocation problem concerns how to satisfy a request of size n from a list of free holes. There are many solutions to this problem.
  - The first-fit
  - The best-fit &
  - The worst-fit
- Strategies are the ones most commonly used to select a free hole from the set of available holes.

Memory Allocation

- **First fit**. Allocate the first hole that is big enough. Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended. We can stop searching as soon as we find a free hole that is large enough.
- **Best fit**. Allocate the smallest hole that is big enough. We must search the entire list, unless the list is ordered by size. This strategy produces the smallest leftover hole.
- **Worst fit**. Allocate the largest hole. Again, we must search the entire list, unless it is sorted by size. This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.

Memory Allocation

- Math example on First fit. Best fit. & Worst fit.

Memory Allocation

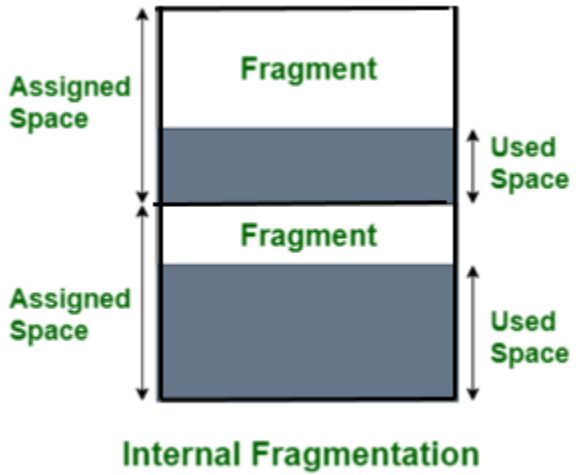
- Simulations have shown that both first fit and best fit are **better** than worst fit in terms of decreasing time and storage utilization.(For fixed size partitioning) Neither first fit nor best fit is clearly better than the other in terms of storage utilization, but first fit is generally faster.
- For variable size partitioning, Best fit performs the **worst**.

Fragmentation

- As processes are loaded and removed from memory, the free memory space is broken into little pieces.
- **Two types of fragmentation:**
  - Internal fragmentation
  - External fragmentation

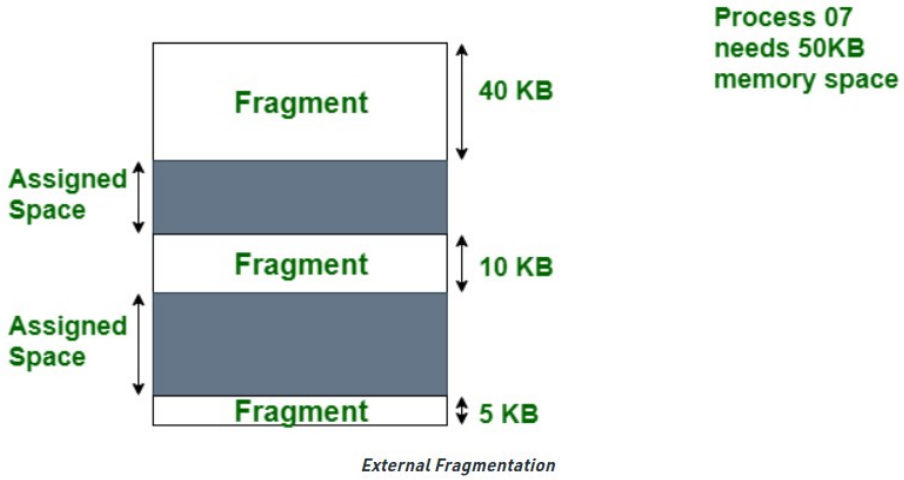
Fragmentation

- Internal fragmentation: Internal fragmentation occurs when memory is divided into fixed-sized partitions. The memory allocated to a process may be slightly larger than the requested memory. The difference between these two numbers is internal fragmentation—unused memory that is internal to a partition



Fragmentation

- External fragmentation: External fragmentation exists when there is enough total memory space to satisfy a request but the available spaces are not contiguous. It doesn't occur in variable size partitioning.
- Storage is fragmented in large number of small holes.



Solution of External Fragmentation

- One solution to the problem of external fragmentation is compaction. The goal is to shuffle the memory contents so as to place all free memory together in one large block. Compaction is not always possible. When compaction is possible, we must determine its cost. The simplest compaction algorithm is to move all processes toward one end of memory; all holes move in the other direction, producing one large hole of available memory. This scheme can be expensive.
- Another possible solution to the external-fragmentation problem is to permit the logical address space of the processes to be noncontiguous, thus allowing a process to be allocated physical memory wherever such memory is available. Two complementary techniques achieve this solution: **segmentation** (Section 8.4) and **paging** (Section 8.5). These techniques can also be combined.

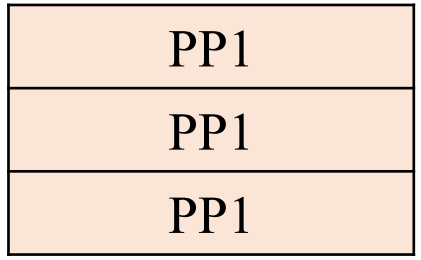
Paging

Basic method of Paging:

- Break physical memory into fixed-sized blocks called frames.
- Breaks logical memory into blocks of the same size called pages.
- When a process is to be executed, its pages are loaded into any available memory frames from the backing store.
- The backing store is divided into fixed-sized blocks that are of the same size as the memory frames.

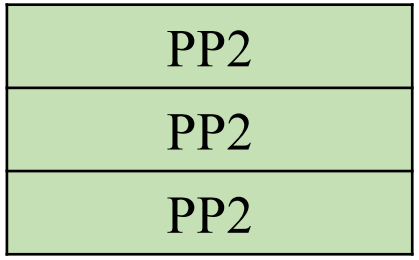
Paging

Process P1

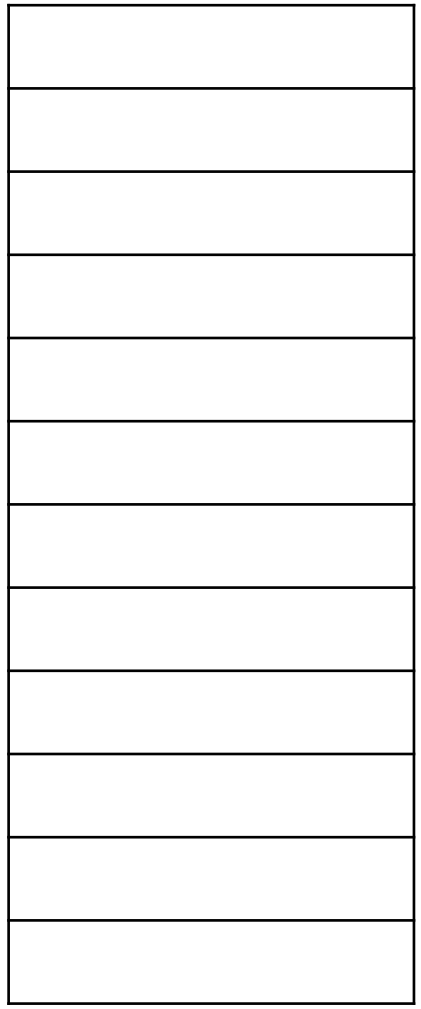


Pages of Process P1

Process P2

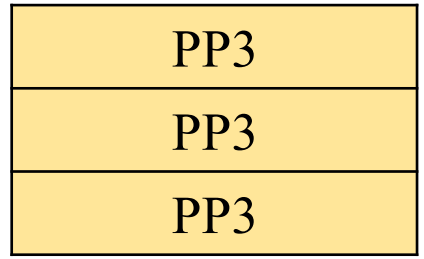


Pages of Process P2



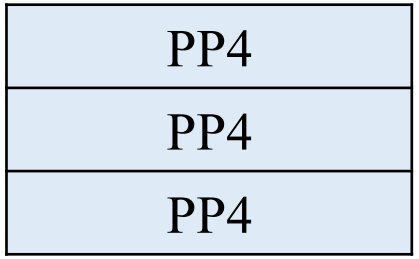
Physical memory divided into frames

Process P3



Pages of Process P3

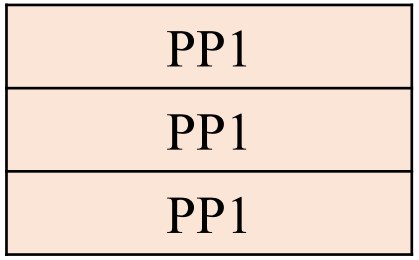
Process P4



Pages of Process P4

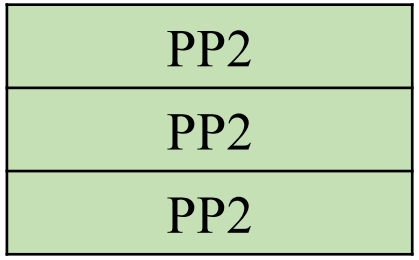
Paging

Process P1

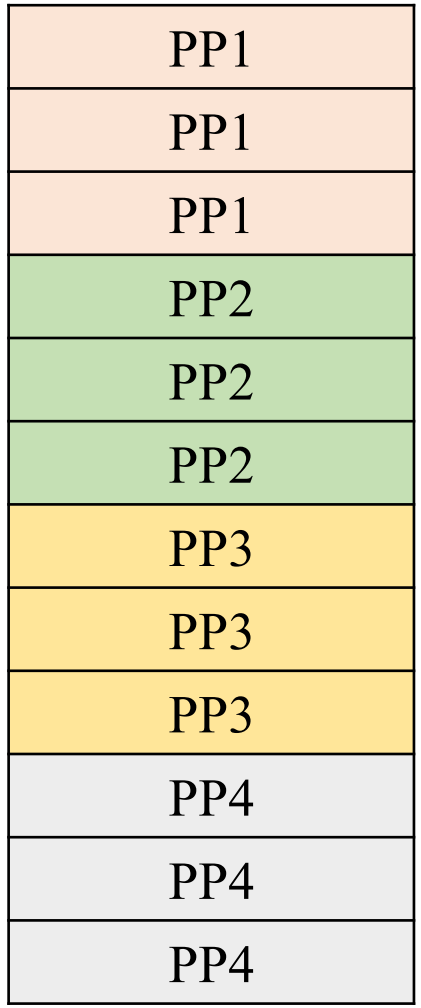


Pages of Process P1

Process P2

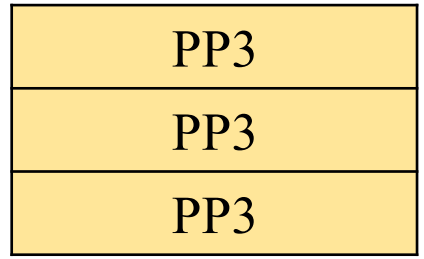


Pages of Process P2



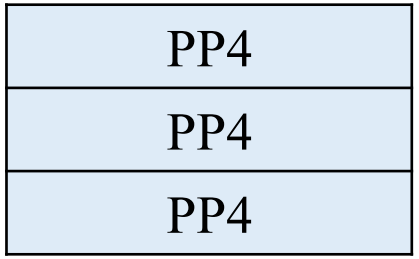
Physical memory divided into frames

Process P3



Pages of Process P3

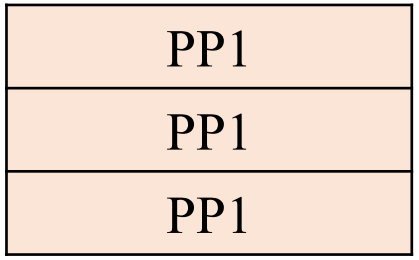
Process P4



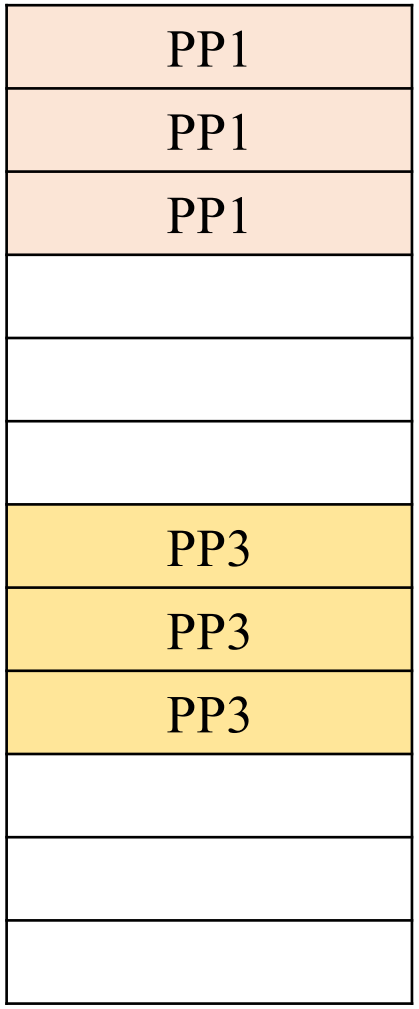
Pages of Process P4

Paging

Process P1

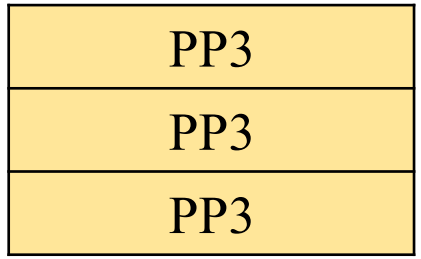


Pages of Process P1



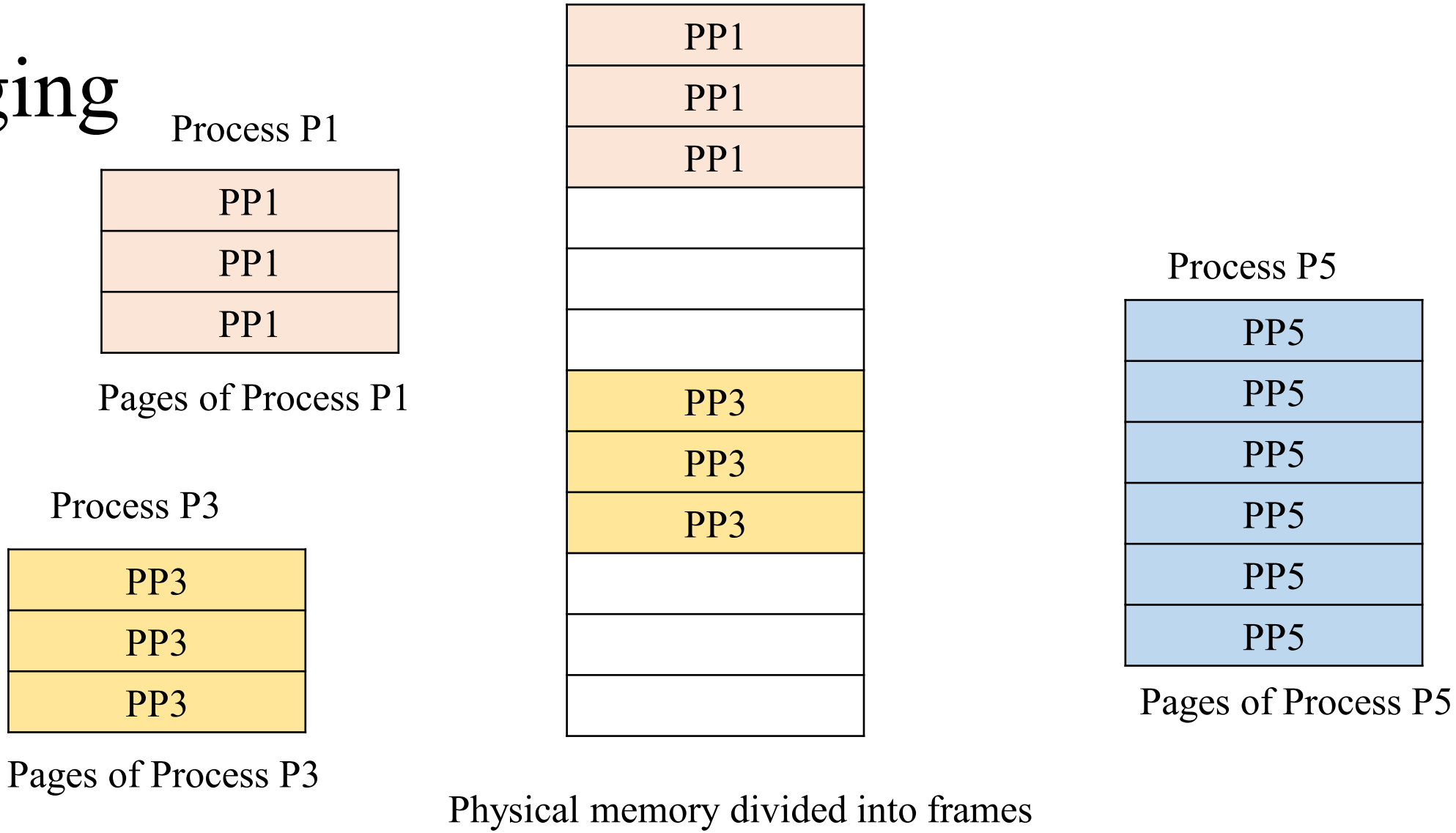
Physical memory divided into frames

Process P3

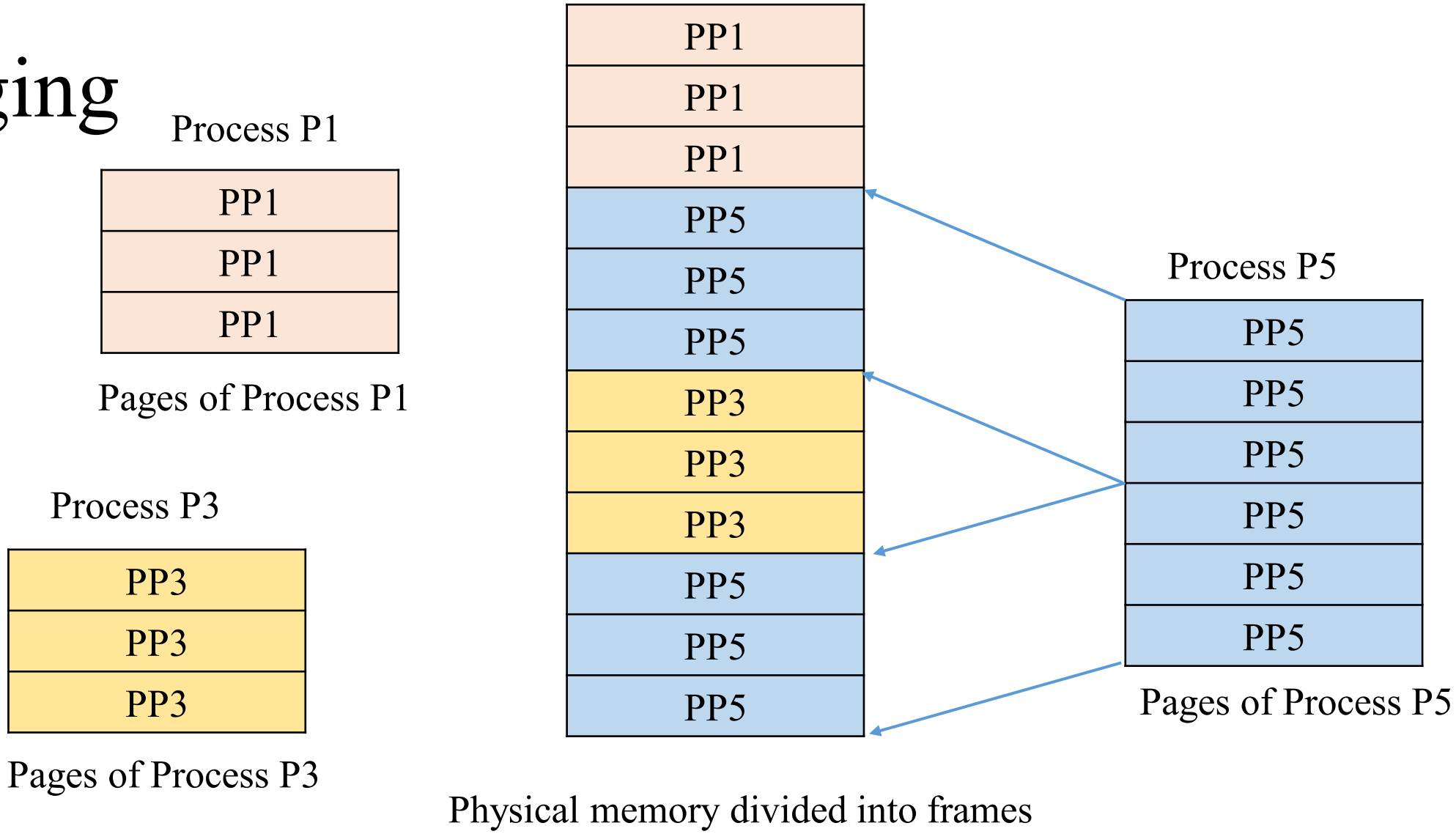


Pages of Process P3

Paging


Process P1
PP1
PP1
PP1
Pages of Process P1
Process P3
PP3
PP3
PP3
Pages of Process P3
PP1
PP1
PP1
PP3
PP3
PP3
Physical memory divided into frames
Process P5
PP5
PP5
PP5
PP5
PP5
PP5
Pages of Process P5

Paging


Process P1
PP1
PP1
PP1
Pages of Process P1
Process P3
PP3
PP3
PP3
Pages of Process P3
PP1
PP1
PP1
PP5
PP5
PP5
PP3
PP3
PP3
PP5
PP5
PP5
Physical memory divided into frames
Process P5
PP5
PP5
PP5
PP5
PP5
PP5
Pages of Process P5

Page table

- Which page belongs to which frame?
- Every address generated by the CPU is divided into two parts: a page number (p) and a page offset (d).

- The page table contains the base address of each page in physical memory.
- This base address is combined with the page offset to define the physical memory address that is sent to the memory unit.

Page table

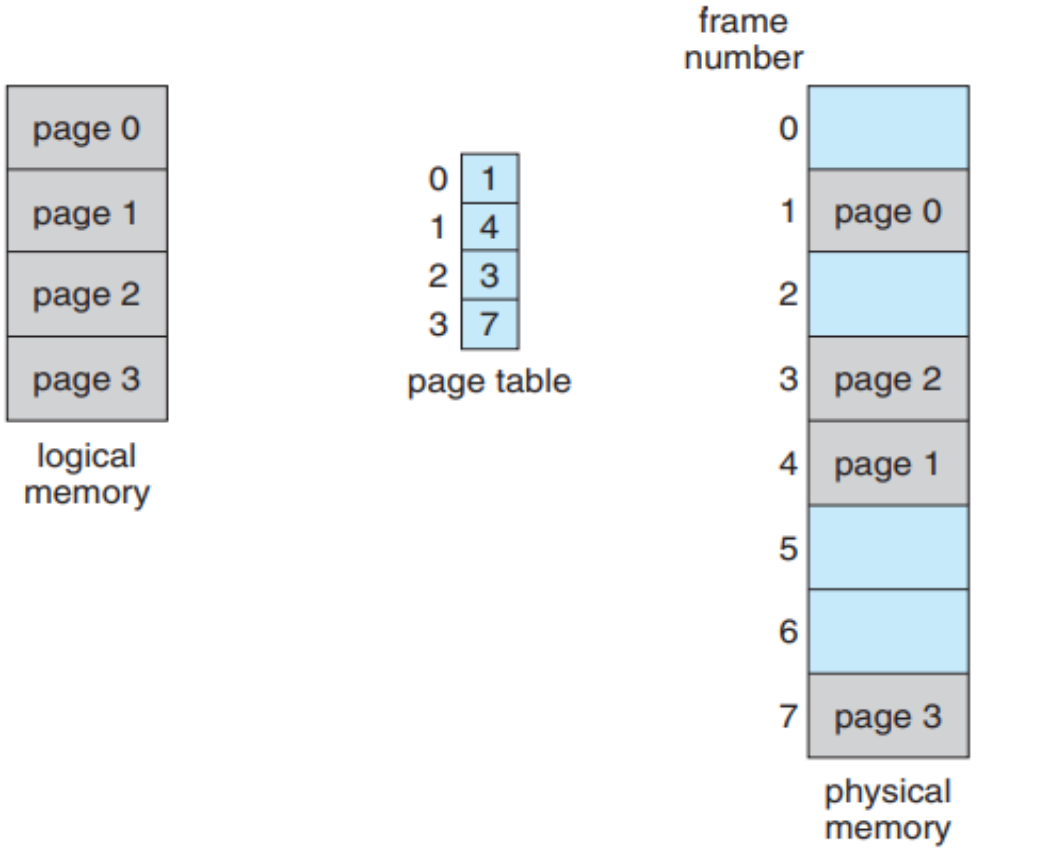


Figure 8.11 Paging model of logical and physical memory.

Page table

Translation of logical address to physical address using Page Table:

- Every address generated by the CPU is divided into two parts:



Where,

p is an index into the page table &

d is the displacement within the page.

Page table

Translation of logical address to physical address using Page Table:

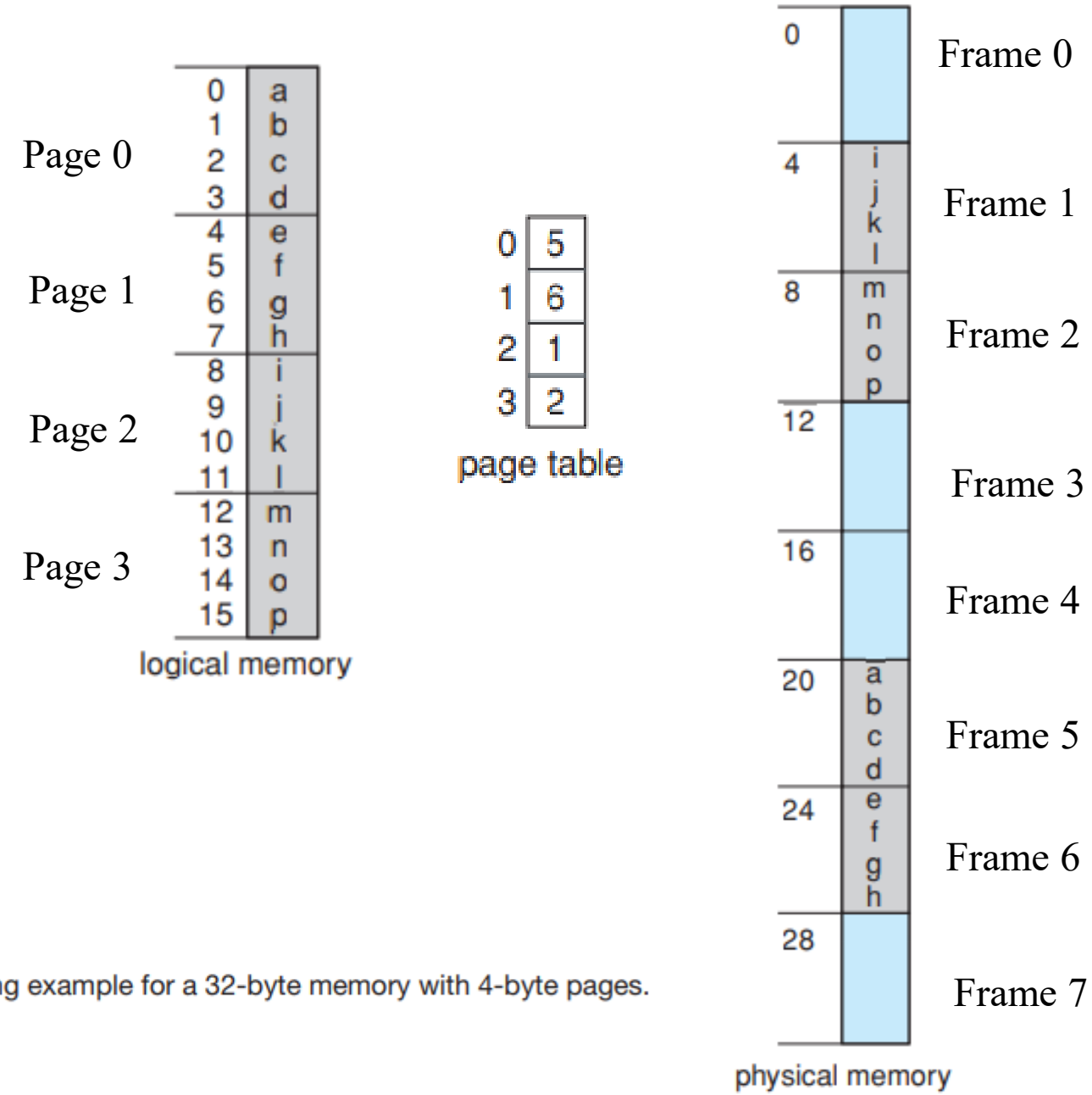


Figure 8.12 Paging example for a 32-byte memory with 4-byte pages.

Page table

Translation of logical address to physical address using Page Table:

- Logical address 0 maps to physical address:

  (Frame*page size+ offset)

  =[(5 × 4) + 0]

  =20

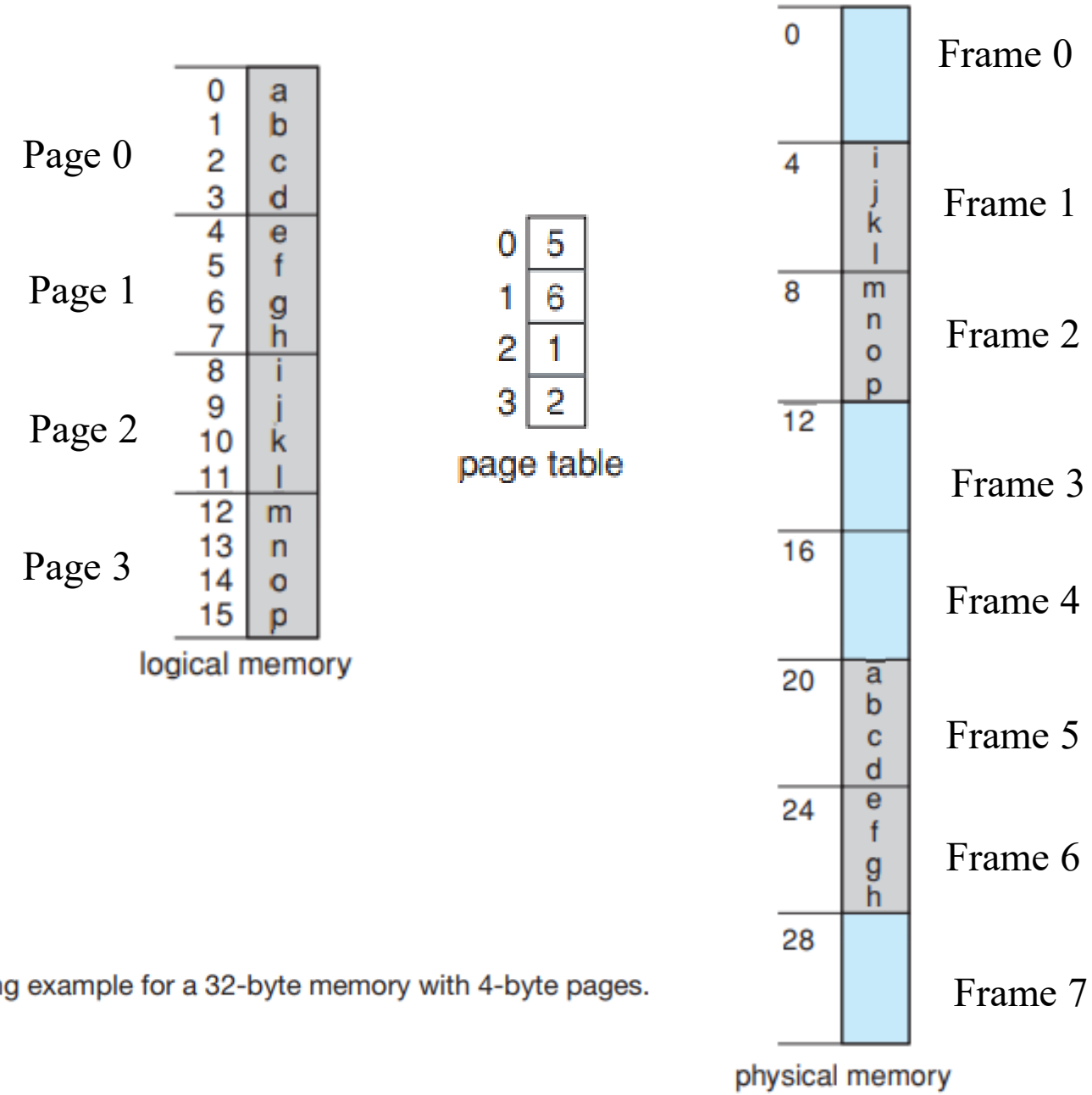


Figure 8.12 Paging example for a 32-byte memory with 4-byte pages.

Page table

Translation of logical address to physical address using Page Table:

- Logical address 3 (page 0, offset 3) maps to physical address:

  (Frame*page size+ offset)

  = (5 × 4) + 3

  =23

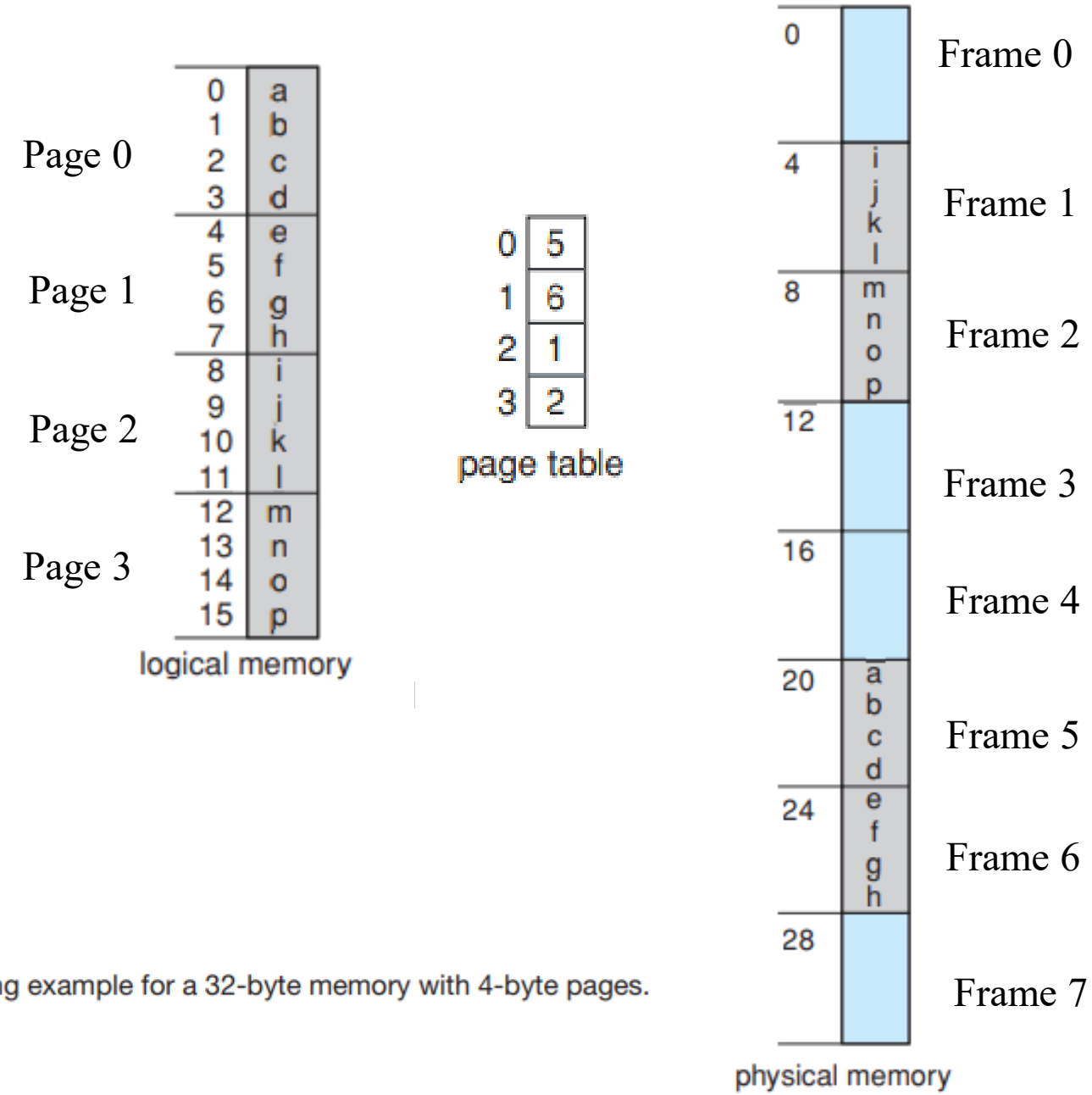


Figure 8.12 Paging example for a 32-byte memory with 4-byte pages.

Page table

- Logical address 4 is page 1, offset 0; according to the page table, page 1 is mapped to frame 6. Thus, logical address 4 maps to physical address 24 [= (6 × 4) + 0].
- Logical address 13 maps to physical address 9.

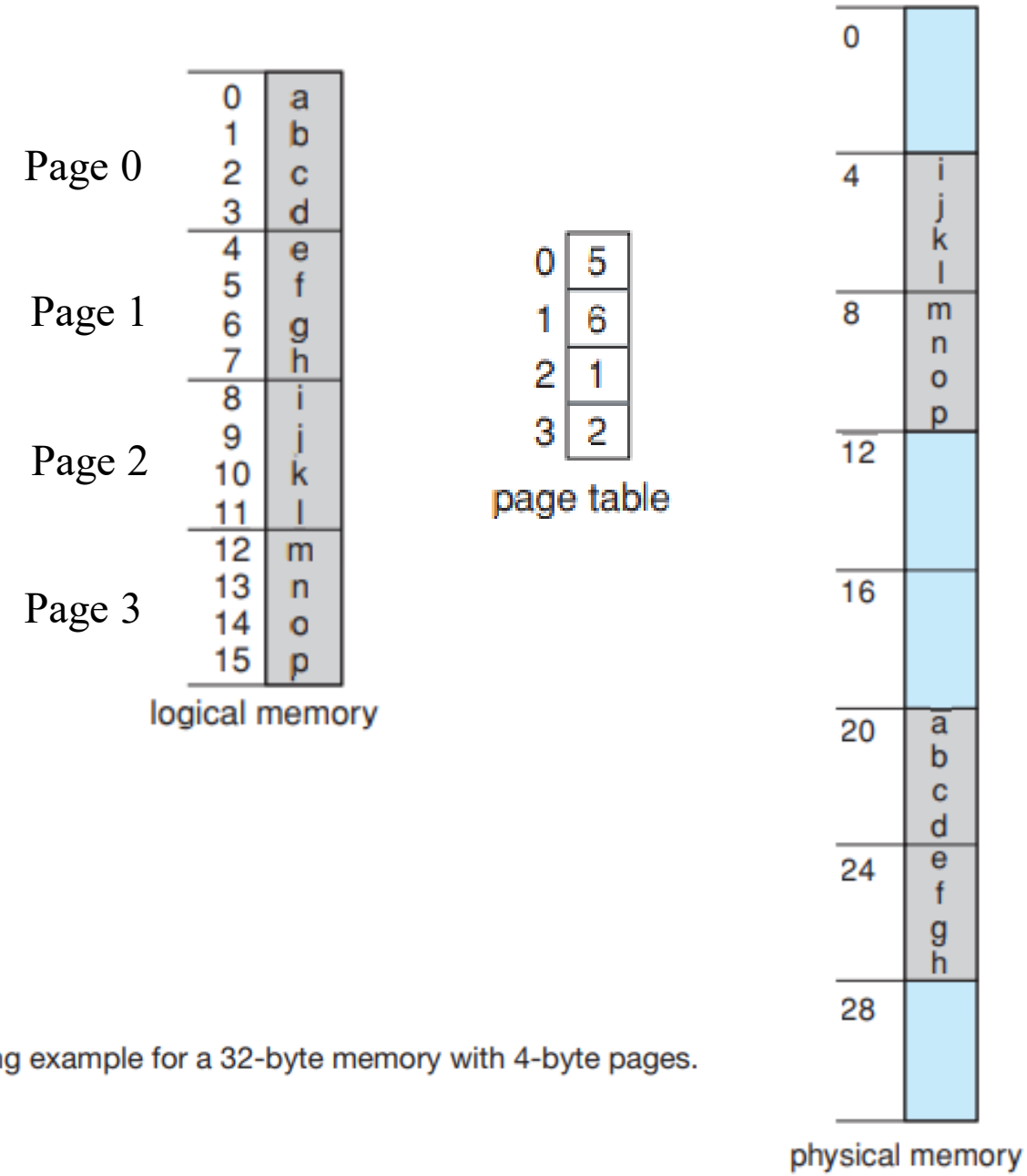


Figure 8.12 Paging example for a 32-byte memory with 4-byte pages.

Hardware Implementation of Page Table

- In the simplest case, the page table is implemented as a set of dedicated registers.

  The use of registers for the page table is satisfactory if the page table is reasonably small (for example, 256 entries).

- The page table is kept in main memory, and a page-table base register (PTBR) points to the page table.

  With this scheme, two memory accesses are needed to access a byte (one for the page-table entry, one for the byte). Thus, memory access is slowed by a factor of 2.

Hardware Implementation of Page Table

- The standard solution to this problem is to use a special, small, fast lookup hardware cache called a **translation look-aside buffer (TLB).**
- The TLB is associative, high-speed memory.
- TLB consists of two parts:
  - key (or tag)
  - value.
- When the associative memory is presented with an item, the item is compared with all keys simultaneously. If the item is found, the corresponding value field is returned.

Hardware Implementation of Page Table

- The TLB contains only a few of the page-table entries.
- When a logical address is generated by the CPU, its page number is presented to the TLB.
- If the page number is found, its frame number is immediately available and is used to access memory. ---**TLB Hit**
- If the page number is not found, a memory reference to the page table must be made.. ---**TLB Miss**
- When the frame number is obtained, we can use it to access memory.
- Also we add the page number and frame number to the TLB, so that they will be found quickly on the next reference.

Hardware Implementation of Page Table

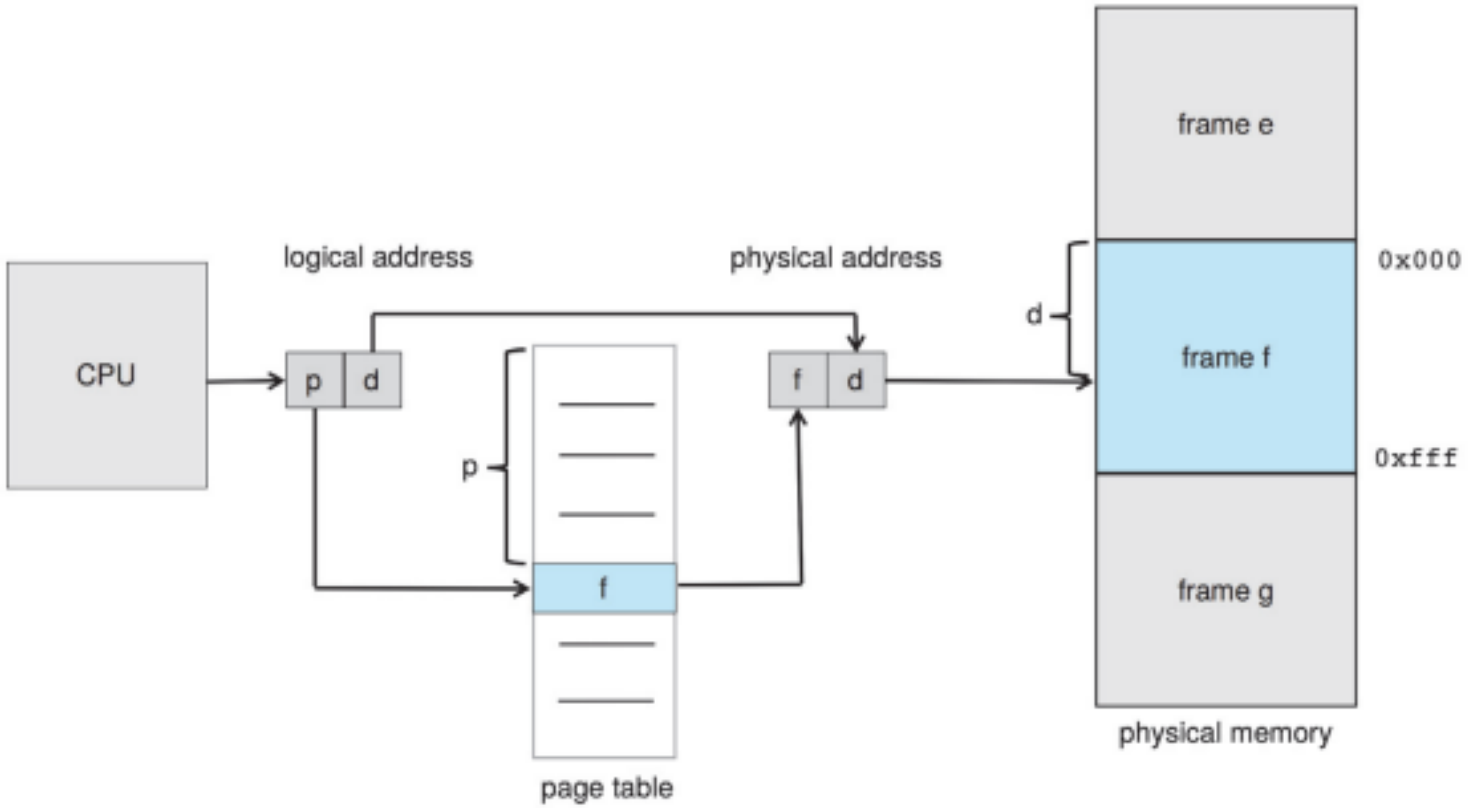


Figure 9.8 Paging hardware.

Hardware Implementation of Page Table

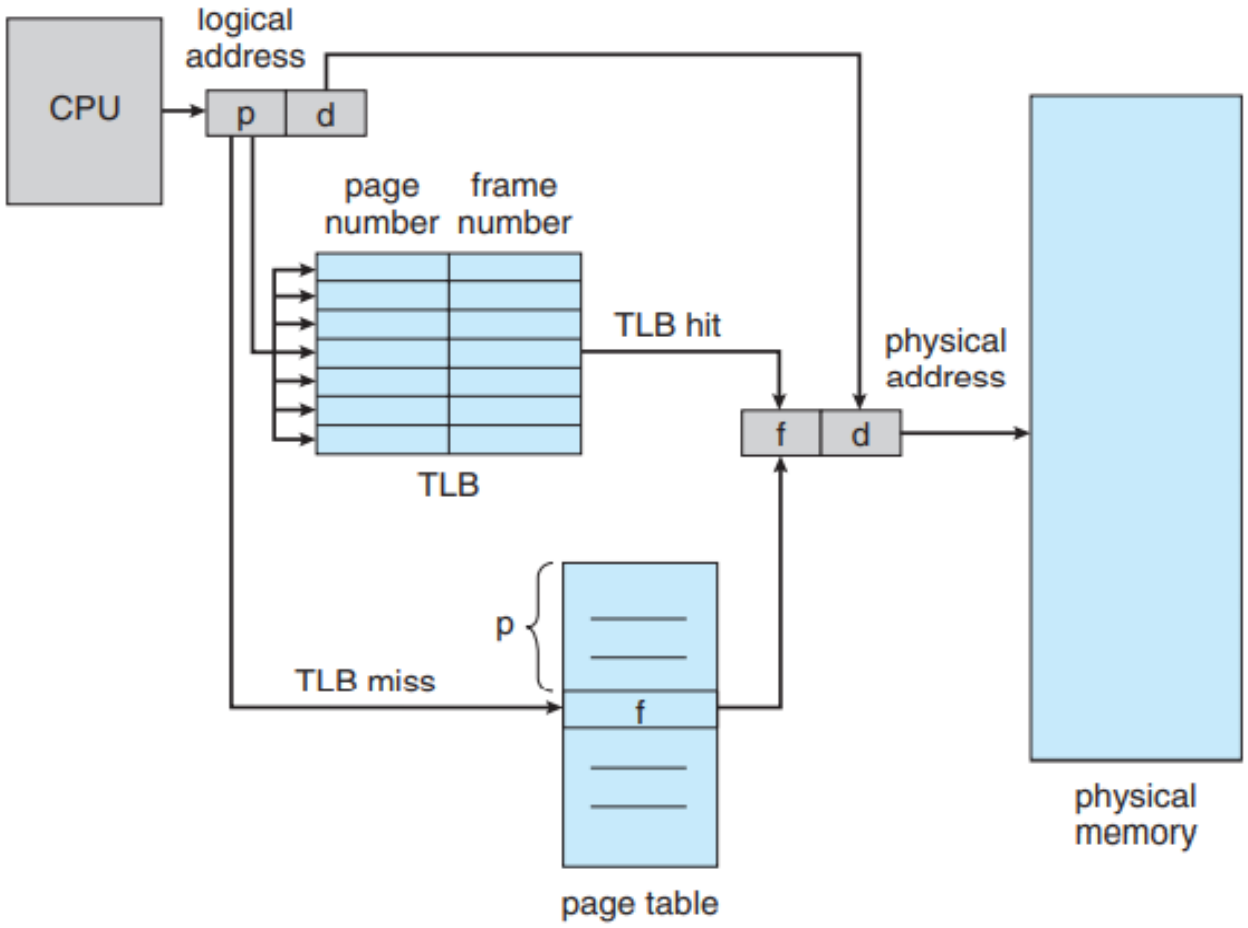
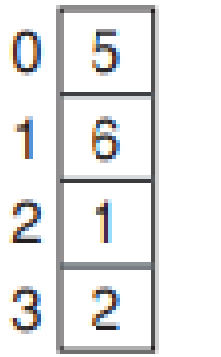


Figure 8.14 Paging hardware with TLB.

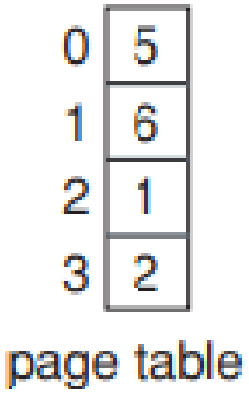
Page Table Entries

- Page table entries contains several information about pages which vary from OS to OS.
- The **most important** information in a page table entry is the **frame number.**
- The remaining information is optional.



page table

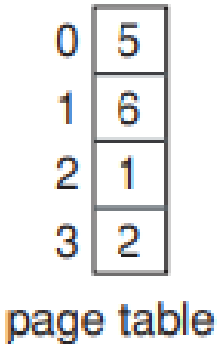
Page Table Entries



Frame number	Present/ Absent	Protection	Reference	Caching	Dirty

- **Frame Number** : it denotes the frame where the page is present in the main memory.
- The number of bits required depends on the number of frames in the main memory.

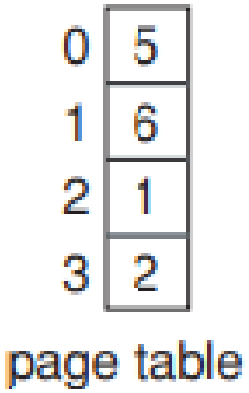
Page Table Entries



Frame number	Present/ Absent	Protection	Reference	Caching	Dirty

- **The Present / Absent** bit specifies whether the page is present in the main memory or not.
- It is also called Valid / Invalid bit.
- If the page we are looking for is not present in main memory, it is called PAGE FAULT.
- If the page we are looking for is not present in main memory, the Present/Absent bit is set to 0.

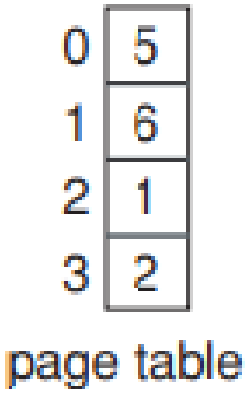
Page Table Entries



Frame number	Present/ Absent	Protection	Reference	Caching	Dirty

- **The Protection bit** also known as Read/Write bit is used for page protection.
- It specifies the permissions for read or write operations on the page.
- The bit is set to 0 if only read operation is allowed.
- The bit is set to 1 if both read and write operations are allowed.

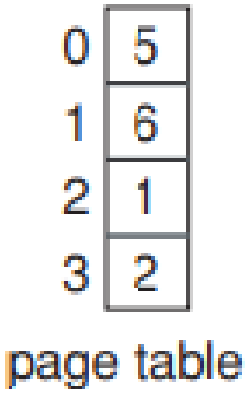
Page Table Entries



Frame number	Present/ Absent	Protection	Reference	Caching	Dirty

- **The Reference bit** specifies whether the page has been referenced in the last clock cycle or not.
- It is set to 1 when the page is accessed.

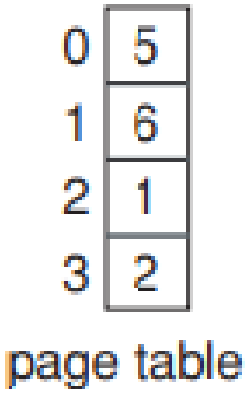
Page Table Entries



Frame number	Present/ Absent	Protection	Reference	Caching	Dirty

- The caching bit is used for enabling or disabling caching of the page.
- When we need fresh data we have to disable caching so as to avoid fetching of old data from the cache.
- When caching has to be disabled, this bit is set to 1. Otherwise **it is set to 0.**

Page Table Entries



Frame number	Present/ Absent	Protection	Reference	Caching	Dirty

- **The Dirty bit** is also known as the Modified bit.
- It specifies whether the page has been modified or not.
- If the page has been modified, then this bit is set to 1 otherwise set to 0.
- This bit helps in avoiding unnecessary writes to the secondary memory when a page is being replaced by another page.

Shared Pages

- An advantage of paging is the possibility of sharing common code.
- This consideration is particularly important in a time-sharing environment.
- Consider a system that supports 40 users, each of whom executes a text editor. If the text editor consists of 150 KB of code and 50 KB of data space, we need 8,000 KB to support the 40 users.

Shared Pages

- If the code is reentrant code (or pure code), however, it can be shared, as shown in Figure 8.16.

  (Code that is not self-modifying code. It never changes during execution)

- Each process has its own copy of registers and data storage to hold the data for the process's execution.
- The data for two different processes will, of course, be different.

Shared Pages

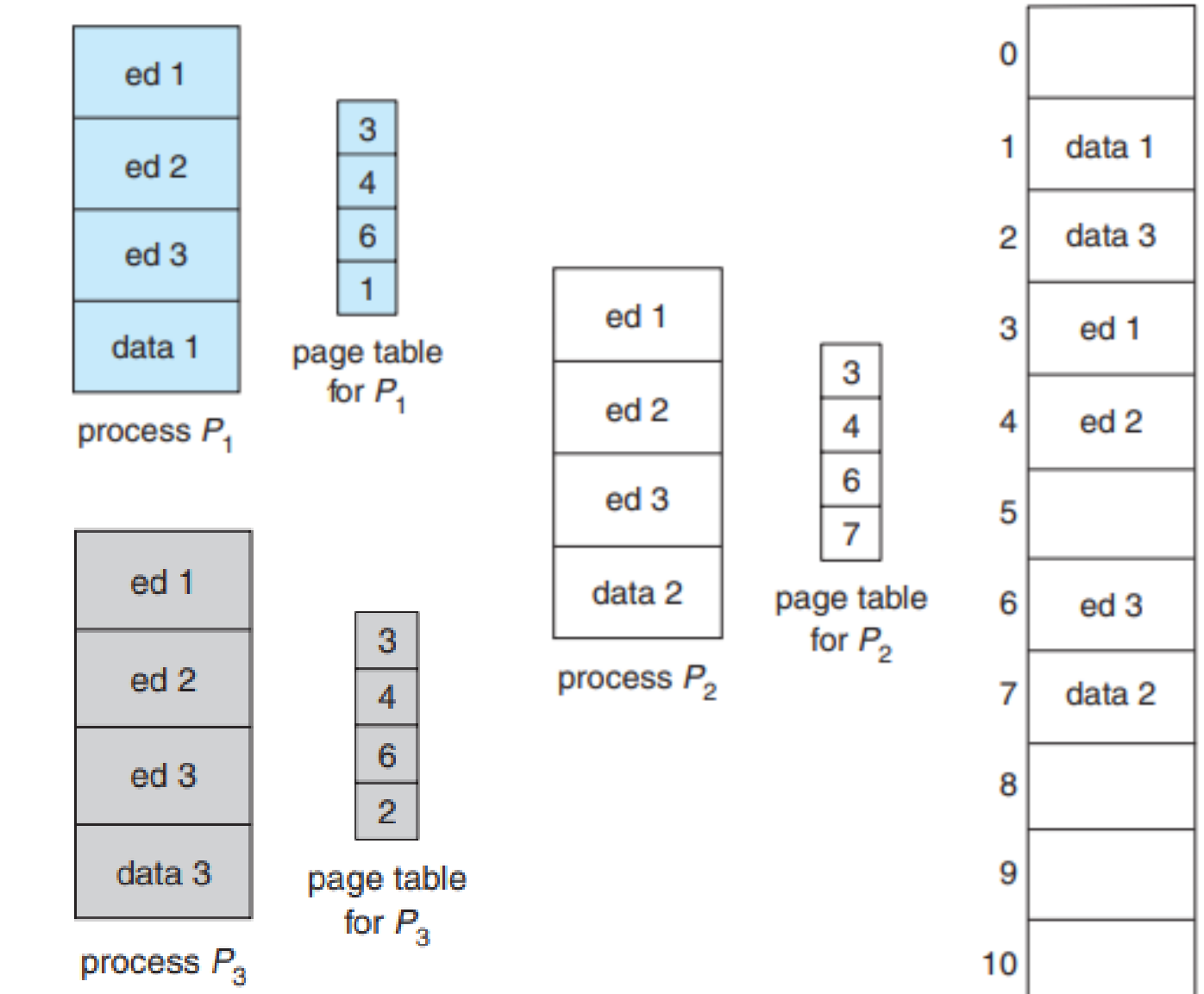


Figure 8.16 Sharing of code in a paging environment.

Shared Pages

- Consider a system that supports 40 users, each of whom executes a text editor. If the text editor consists of 150 KB of code and 50 KB of data space, we need 8,000 KB to support the 40 users.
- Now if code is shared, we need:

  =150+(50*40)

  =150+2000

  =2150 KB to support 40 users.

Structure of the Page Table

- ❖Hierarchical Page Table/ Multi level Page Table
- ❖Hashed Page Tables
- ❖Inverted Page Table

Hierarchical Paging

- Most modern computer systems support a large logical address space. (2^{32} to 2^{64}).
- In such an environment, the page table itself becomes excessively large.

For example, consider a system with:

Physical address space(M.M)= 256MB

Logical address space(S.M)= 4GB

Frame size=4KB

Page table entry=2B

Hierarchical Paging

For example, consider a system with:

Physical address space(M.M)= 256MB= $\mathbf{2^{28}}$

Logical address space(S.M)= 4GB= $\mathbf{2^{32}}$

Frame size=4KB=$\mathbf{2^{12}}$

Page table entry=2B

So, we can not allocate 2MB size page table in 4KB frame.

Hierarchical Paging

- Most modern computer systems support a large logical address space. ($\mathbf{2^{32}}$ to $\mathbf{2^{64}}$).

Another example, consider a system with a 32-bit logical address space. If the page size in such a system is 4 KB ($\mathbf{2^{12}}$),

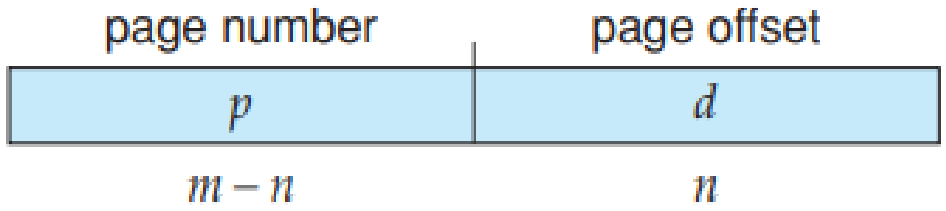
- Then a page table may consist of up to 1 million entries ($\mathbf{2^{20}}$). Assuming that each entry consists of 4 bytes,
- Each process may need up to (4 bytes* 1 million)= 4 MB of physical address space for the page table alone.

Hierarchical Paging

- Trying to allocate a page table of this size contiguously in main memory is clearly not a good idea.
- One simple solution to this problem is to divide the page table into smaller pieces. We can accomplish this division in several way.
- One way is to use a two-level paging algorithm, in which the page table itself is also paged (Figure 8.17).

Hierarchical Paging

Generally Logical address is represented like:



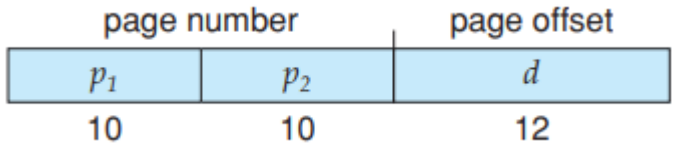
Where,

p is an index into the page table &

d is the displacement within the page.

Hierarchical Paging

For multi level paging, logical address can be divided like this:



Where,

p1 is an index into the outer page table and

p2 is the displacement within the page of the inner page table.

d is the displacement within the page.

Hierarchical Paging

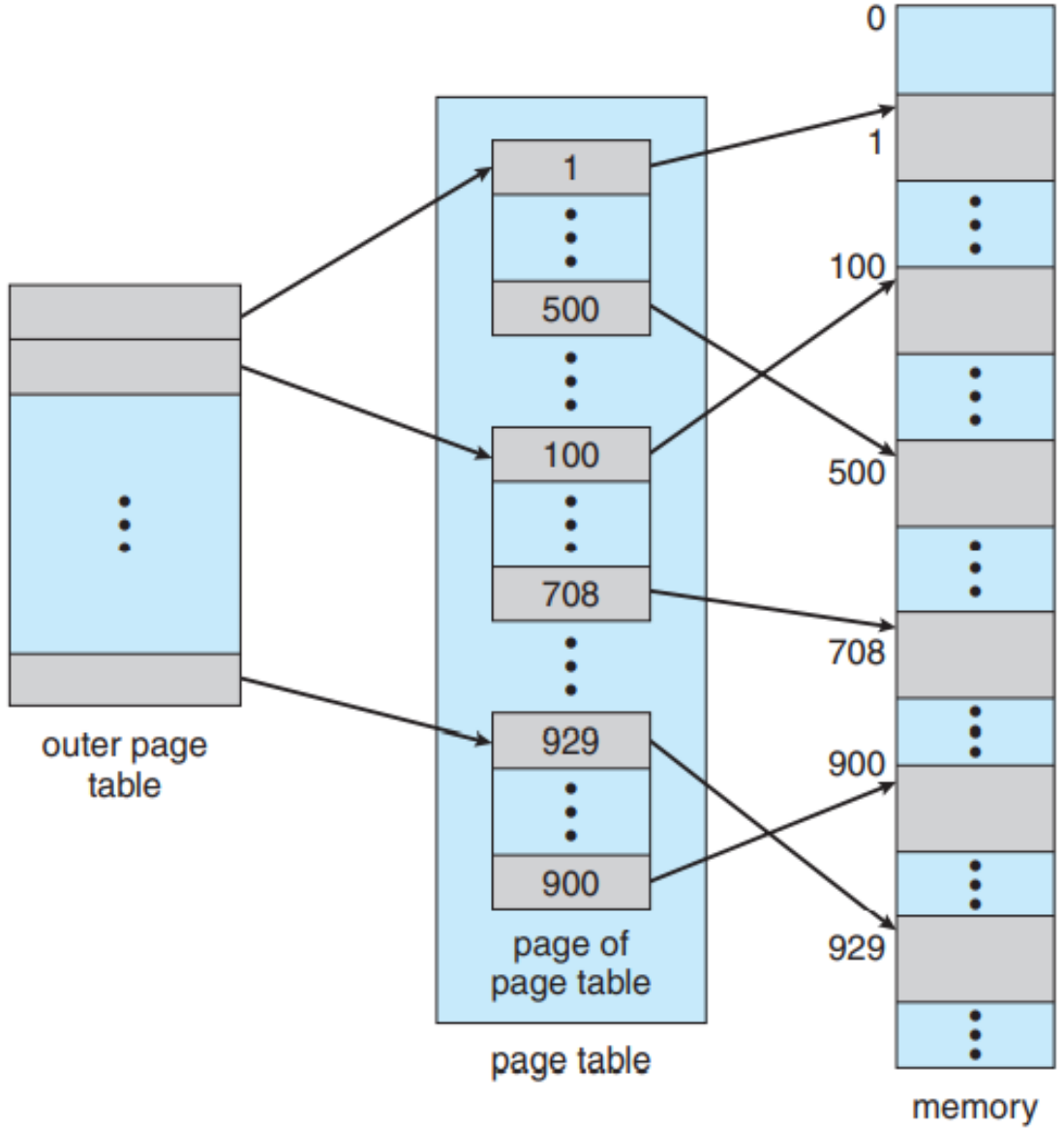


Figure 8.17 A two-level page-table scheme.

Hierarchical Paging

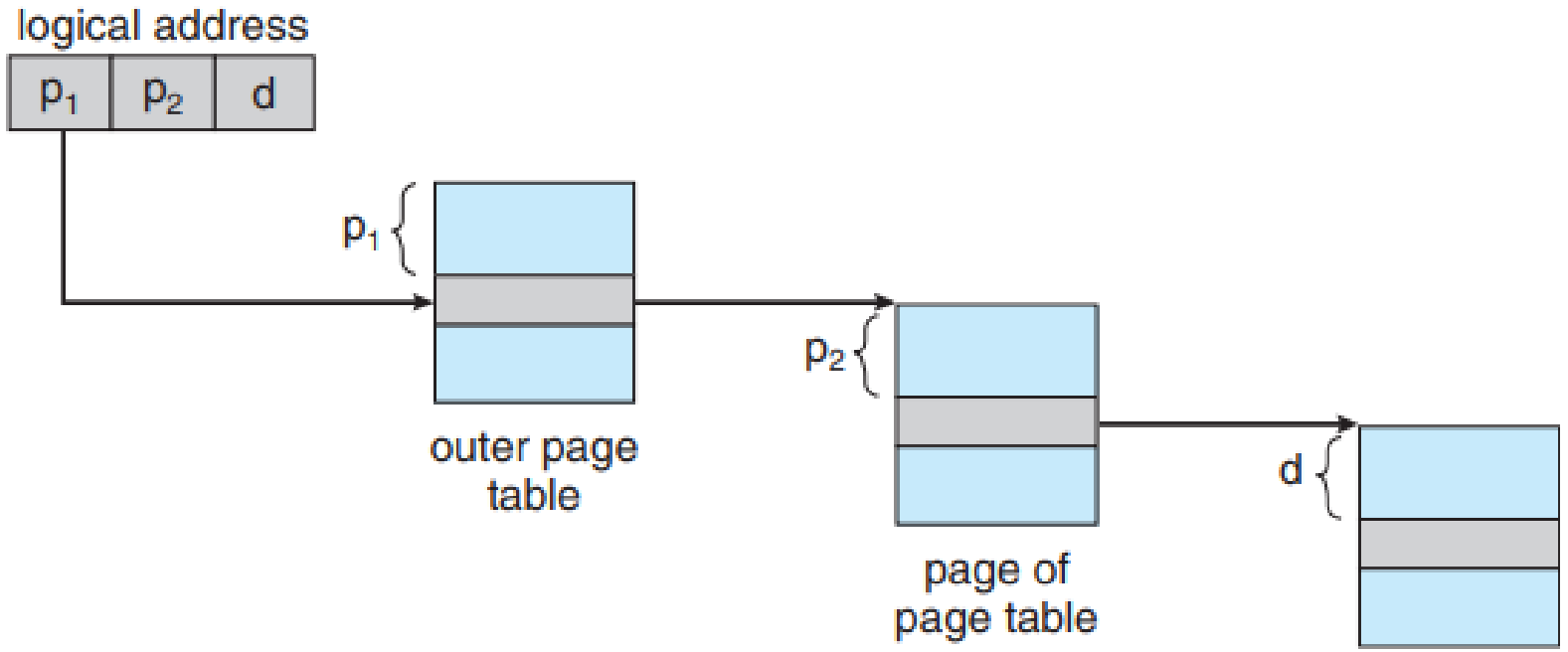


Figure 8.18 Address translation for a two-level 32-bit paging architecture.

Hashed Page Table

- A common approach for handling address spaces larger than 32 bits is to use a hashed page table.
- Here,
  - The hash value being the virtual page number.
  - Each entry in the hash table contains a linked list of elements that hash to the same location (to handle collisions).

Hashed Page Table

- Each element of link list consists of three fields:
  - ❑the virtual page number
  - ❑the value of the mapped page frame
  - ❑a pointer to the next element in the linked list.

Virtual page number	The value of the mapped page frame	A pointer to the next element in the linked list.

Hashed Page Table

- Virtual page number will be passed on the hash function.
- The hash function will be able to find out which entry it need to search(which entry contains the virtual page number p).
- Each entry of the hash table contains the linked list (containing three elements).

Hashed Page Table

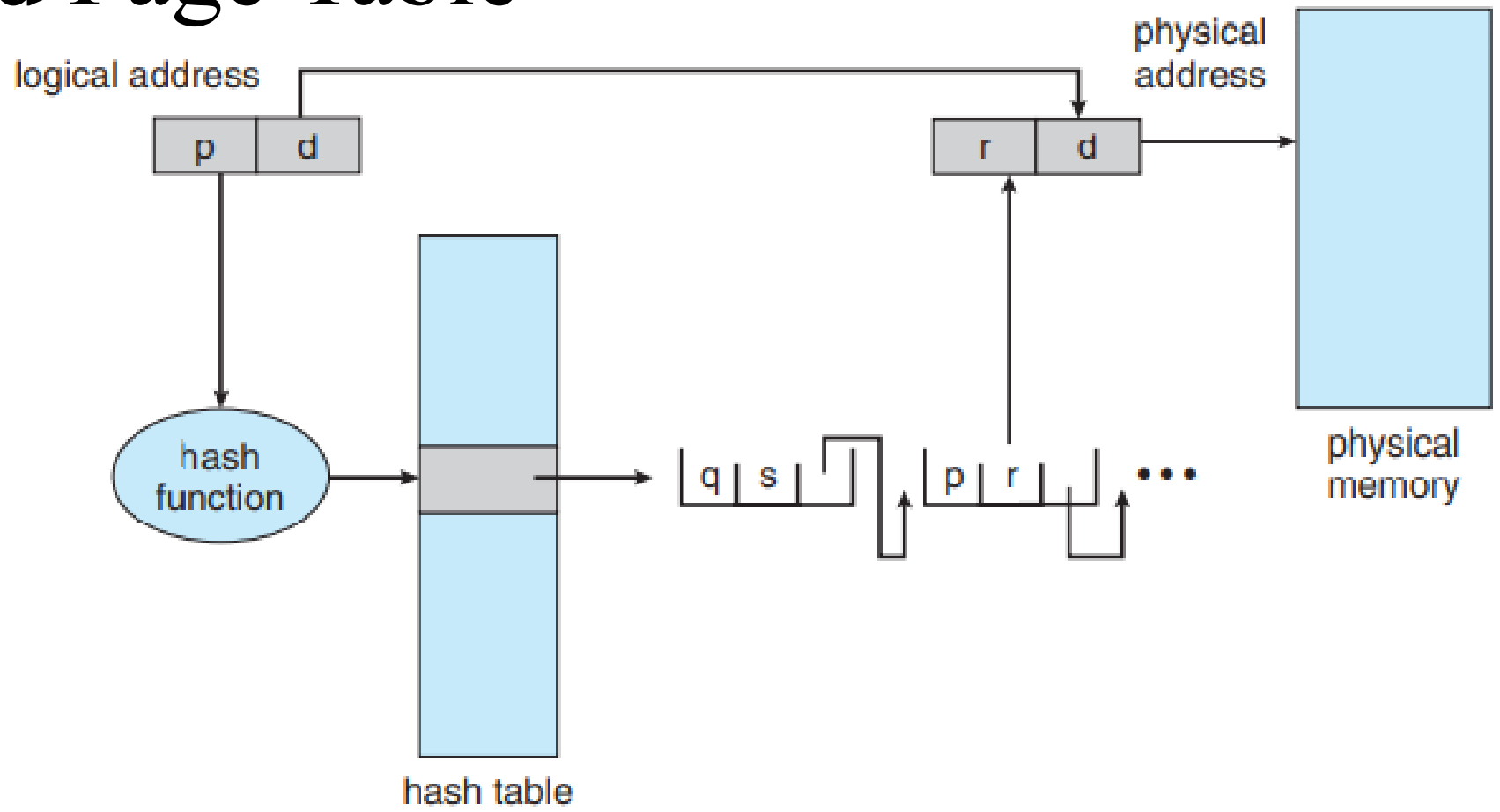


Figure 8.19 Hashed page table.

Inverted Page Table

- Usually, each process has a separate page table.
- For n number of pages there will be n number of page tables.
- For large processes, there would be many pages and for maintaining information about these pages, there would be to many entries in their page tables which itself would occupy a lot of memory.
- Hence memory utilization is not efficient as a lot of memory is wasted in maintaining page tables itself.

Inverted Page Table

- Solution: Inverted Page Table
- An inverted page table has one entry for each real page (or frame) of memory



A page that is present in the main memory.

- Each entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns the page.

Inverted Page Table

- Each virtual address in the system consists of a triple:

Process ID	Page number	Offset

- Each inverted page table entry is a pair:

Process ID	Page number

Inverted Page Table

The Working:

- When a memory reference occurs, part of the virtual address, consisting of <process-id, page number>, is presented to the memory subsystem.
- The inverted page table is then searched for a match.
- If a match is found-say at entry 'i', then the physical address <i, offset> is generated.
- If no match is found, then an illegal address access has been attempted.

Inverted Page Table

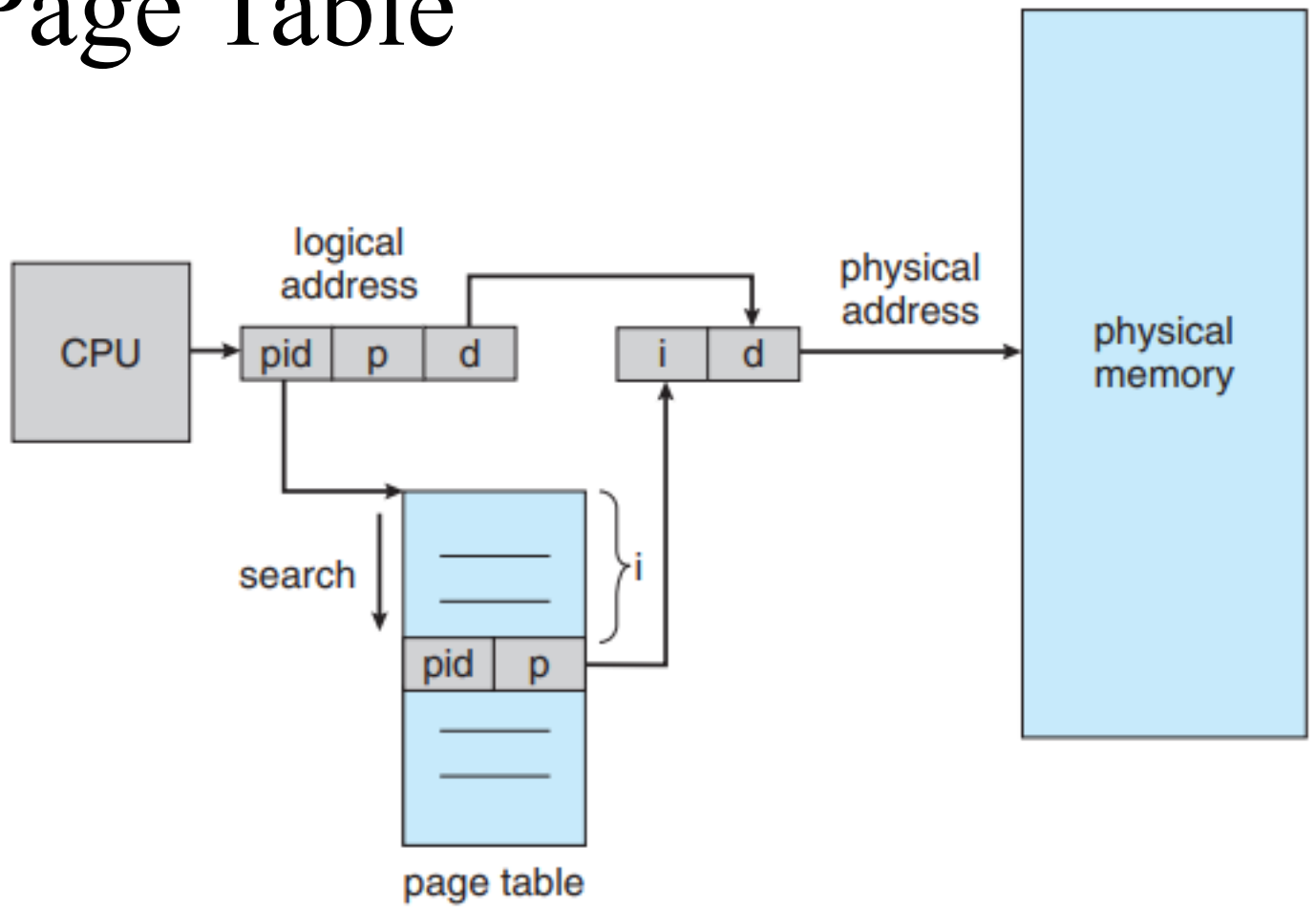


Figure 8.20 Inverted page table.

Advantages of Inverted Page Table

- Advantages of Inverted Page Table
  - Reduces memory usage
- Disadvantages of Inverted Page Table
  - Increases search time as the table when a page reference occurs. Because the inverted page table is sorted by physical address, but lookups occur on virtual addresses, the whole table might need to be searched before a match is found. This search would take far too long.
  - Difficulty in implementing shared memory.